SEO PACKAGE

1. SEO Title

Maximum Product of Two Elements in an Array — LeetCode 1464 Python Solution Explained (Sort vs Single Pass)

2. SEO Subtitle

Learn the "Top-Two Grab" pattern that turns a scary-looking array problem into a clean two-line Python solution — and why `[2,2]` correctly returns `1`.

3. Featured Quote

> "The whole trick is realizing small numbers can never help you — you only ever need the two largest."

4. Meta Description

Solve LeetCode 1464 Maximum Product of Two Elements in Python. Sort vs single-pass, complexity, edge cases, and why [2,2] returns 1 — beginner friendly.

5. URL Slug

`maximum-product-of-two-elements-in-an-array-leetcode-python`

6. Keywords

- maximum product of two elements in an array
- LeetCode 1464
- LeetCode Python solution
- coding interview array problems
- Python array problem beginner

- find two largest numbers Python
- top two elements algorithm
- sorting vs single pass
- O(n log n) vs O(n)
- time complexity beginners
- Python interview questions
- data structures and algorithms
- easy LeetCode problems
- array manipulation Python
- coding interview preparation
- software engineering fundamentals
- Python one-liner solution
- max product array
- beginner coding tutorial
- LeetCode easy Python

7. Tags

`Python` · `LeetCode` · `Coding Interview` · `Algorithms` · `Data Structures` · `Arrays` · `Sorting` · `Time Complexity` · `Beginner Coding` · `Software Engineering` · `Problem Solving` · `Interview Preparation` · `LeetCode Easy`

Maximum Product of Two Elements in an Array — The Beginner-Friendly Deep Dive

8. Introduction

Some problems *look* intimidating because of how they're written, not because of how hard they actually are. **Maximum Product of Two Elements in an Array** (LeetCode 1464) is a perfect example. The phrasing — "return the maximum value of `(nums[i] - 1) * (nums[j] - 1)`" — has just enough notation to make a newcomer freeze. But once you strip the wrapping paper off, this is one of the gentlest problems in the entire array category.

So why should you care about a problem this easy?

Because interviewers at companies like **Amazon** and **Google** love using warm-up questions exactly like this. They're not testing whether you can find two numbers. They're testing whether you can *recognize a pattern*, choose between a simple solution and an optimal one, reason about **time and space complexity**, and handle the sneaky edge cases without panicking. Those are the real skills that carry over to every harder problem you'll ever face.

There's also a bigger idea hiding here: **"find the top few of something."** Leaderboards, best-two prices, highest-rated products, top scorers — this instinct shows up constantly in real **software engineering**. Learn it cleanly once, and you'll spot it everywhere.

This article is the first step in a beginner-friendly series. By the end, you'll understand not just *how* to solve it, but *why* every line works.

9. Problem Overview

Here's the problem in plain English.

You're given an array (a list) of integers called `nums`. Your job is to pick **two different positions** in that list. Take the value at each position, subtract `1` from each, and multiply the two results together. Out of every possible pair you could choose, return the **largest** product.

Formally, you want to maximize:

```
(nums[i] - 1) * (nums[j] - 1)
```

where `i` and `j` are two **different indices** (positions), not necessarily two different *values*.

That last sentence is the one detail people miss, so keep it in your back pocket: the two spots must be different, but the numbers sitting in them are allowed to be identical.

The constraints are friendly: - The array always has at least **2** elements. - Every number is a positive integer (typically `1 ≤ nums[i] ≤ 1000`).

10. Example

Let's make it concrete.

Example 1

```
Input:  nums = [3, 4, 5, 2]
```

```
Output: 12
```

Why? The two biggest numbers are `5` and `4`. Subtract one from each to get `4` and `3`. Multiply: `4 * 3 = 12`. No other pair beats that, because feeding in smaller numbers only shrinks the result.

Example 2

```
Input:  nums = [1, 5, 4, 5]
Output: 16
```

Here the two largest are `5` and `5`. Subtract one from each: `4` and `4`. Multiply: `4 * 4 = 16`.

Example 3 (the sneaky one)

```
Input:  nums = [2, 2]
Output: 1
```

Both numbers are `2`. Subtract one from each: `1` and `1`. Multiply: `1 * 1 = 1`. Even though the values are identical, they sit in **different positions**, so the pair is valid. This is the `[2,2]` case people trip over — the answer is `1`, not `4`.

11. Intuition

Let's build the insight the way an experienced engineer actually would — slowly, without jumping to code.

Start by asking: *what does the formula reward?* We take each chosen number, knock it down by one, then multiply. Multiplication of two positive numbers grows when either number grows. Subtracting one first doesn't change that direction — a bigger input still produces a bigger `(x - 1)`.

That single observation is the whole game:

> **Bigger inputs always produce a bigger result.**

If that's true, then the smallest numbers in the array can *never* be part of the best answer. They're dead weight. The only two values that could possibly matter are the **two largest**.

I like to give this a friendly name so it sticks: the **Top-Two Grab**. Whenever a problem's answer depends only on the largest couple of elements, this instinct kicks in.

Now, how do we actually *get* the two largest? The most intuitive trick is **sorting**. Sorting rearranges the numbers from smallest to biggest, like kids lining up shortest to tallest for a class photo. Once

they're lined up, the two tallest are simply **the last two in line**. Grab them, do the math, done.

That's the entire solution in one mental image. Everything below is just turning that picture into code.

12. Brute Force Solution

Before the clever version, let's honor the honest first idea — because it's correct, and correctness comes first.

Idea Try *every* possible pair of positions. Compute `(nums[i] - 1) * (nums[j] - 1)` for each pair. Remember the biggest result you've seen.

Algorithm 1. Set `best = 0`. 2. Loop `i` over every index. 3. Loop `j` over every index *after* `i` (so we never reuse the same position and never double-count a pair). 4. Compute the product for that pair and update `best` if it's larger. 5. Return `best`.

```
def max_product_brute(nums):
    best = 0
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            product = (nums[i] - 1) * (nums[j] - 1)
            best = max(best, product)
    return best
```

Advantages - Dead simple to reason about — it literally checks everything. - Impossible to get wrong logically; it's easy to *believe* it's correct. - A great safety net when you're unsure of a cleverer approach.

Disadvantages - It wastes enormous effort checking pairs we already know are useless (like the two smallest numbers). - It scales poorly as the array grows.

Complexity - **Time:** `O(n²)` — every element paired with every other. - **Space:** `O(1)` — just one running variable.

13. Optimal Solution

Here's the payoff for understanding the Top-Two Grab.

We don't need every pair. We need the two largest numbers. So:

1. **Sort** the array (smallest to largest).
2. Grab the **last** element and the **second-to-last** element.

3. Subtract one from each.
4. Multiply.

That's the whole thing — two lines of real work.

Let's visualize sorting `[3, 4, 5, 2]`:

Step	Array state	Largest two
Original	`[3, 4, 5, 2]`	not obvious yet
After sort	`[2, 3, 4, 5]`	`4` and `5` (last two)
Subtract one	—	`3` and `4`
Multiply	—	**12**

In Python, negative indices count backward from the end:

Index	Meaning	Value in `[2, 3, 4, 5]`
`nums[-1]`	last item	`5`
`nums[-2]`	second-to-last item	`4`

So after sorting, `nums[-1]` and `nums[-2]` are *guaranteed* to be the two biggest numbers — exactly what the Top-Two Grab wants. No magic, just counting from the back.

14. Python Code

```
from typing import List


class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        # Sort ascending, so the two largest values land at the end.
        nums.sort()
        # Take the last two, subtract one from each, multiply.
        return (nums[-1] - 1) * (nums[-2] - 1)
```

And the no-sorting, single-pass version for when you want raw speed:

```
from typing import List


class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        first = second = 0  # largest and second-largest seen so far

        for num in nums:
            if num > first:
```

```
                second = first   # old max becomes second
                first = num      # new max
            elif num > second:
                second = num     # only beats the runner-up

        return (first - 1) * (second - 1)
```

15. Code Walkthrough

Let's break down the **sort-based** solution line by line, since it's the one you'll reach for in most interviews.

- `nums.sort()` rearranges the list *in place* from smallest to largest. After this line, the biggest numbers are parked at the end.
- `nums[-1]` is the largest value; `nums[-2]` is the second largest. Subtracting `1` from each applies the problem's formula.
- The multiplication returns the final answer directly — no extra variables, no loops.

Now the **single-pass** version, which needs a little more care:

- `first` and `second` track the largest and second-largest values seen so far, both starting at `0` (safe, since all inputs are positive).
- For each `num`, if it beats the current `first`, then the *old* `first` gets demoted to `second`, and `num` becomes the new `first`. The order of these two assignments matters — demote before you overwrite.
- The `elif` handles the case where `num` isn't the new maximum but is still bigger than the current runner-up.
- After one pass, we apply the same `(x - 1)` formula.

This is why, for an easy problem, many engineers just sort: the single-pass swap logic is genuinely easy to get subtly wrong.

16. Dry Run

Let's trace the **single-pass** version on `nums = [3, 4, 5, 2]`.

Step	`num`	Condition met	`first`	`second`
Start	—	—	0	0
1	3	`3 > 0` → new max	3	0
2	4	`4 > 3` → new max	4	3
3	5	`5 > 4` → new max	5	4

Step	`num`	Condition met	`first`	`second`
4	2	`2 > 5` ? no. `2 > 4` ? no	5	4

After the loop: `first = 5`, `second = 4`.

Final answer: `(5 - 1) * (4 - 1) = 4 * 3 = 12`. ✅

Now the famous `[2, 2]` case with the **sort** version: - After `sort()`: `[2, 2]`. - `nums[-1] = 2`, `nums[-2] = 2`. - `(2 - 1) * (2 - 1) = 1 * 1 = 1`. ✅

The duplicate values are fine because they occupy two different positions.

17. Complexity Analysis

Sort-based solution - **Time:** `O(n log n)` — dominated entirely by the sort. The index lookups afterward are `O(1)`. - **Space:** `O(1)` extra if you sort in place (Python's `sort()` mutates the list). If you must preserve the original order, `sorted()` costs `O(n)` extra space.

Single-pass solution - **Time:** `O(n)` — one walk through the list, touching each element once. - **Space:** `O(1)` — just two tracking variables.

The single pass is theoretically faster because it avoids the `log n` factor and never rearranges the array. In practice, on the small inputs LeetCode gives you (a few hundred numbers), both finish instantly — so readability wins.

18. Alternative Solutions

A clean middle-ground uses Python's built-in `heapq.nlargest`, which grabs the top *k* elements without a full sort:

```
import heapq
from typing import List


class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        a, b = heapq.nlargest(2, nums)
        return (a - 1) * (b - 1)
```

Comparison:

Approach	Time	Space	Readability
Brute force	`O(n²)`	`O(1)`	Simple but slow
Sort	`O(n log n)`	`O(1)`	Cleanest
Single pass	`O(n)`	`O(1)`	Fast, easy to misorder
`heapq.nlargest`	`O(n log k)`	`O(k)`	Very expressive

For `k = 2`, `nlargest` behaves essentially like the single pass but hides the tricky swap logic behind a well-tested library call — a nice compromise.

19. Edge Cases

- **Smallest possible array (`[a, b]`):** Only one pair exists, and every approach uses it directly. Nothing breaks.
- **All ones (`[1, 1, 1]`):** `1 - 1 = 0`, so the product is `0`. Handled automatically — no special case needed.
- **Duplicates (`[5, 5]` or `[2, 2]`):** Two different *positions* holding the same *value* is legal. Returns `16` and `1` respectively.
- **Maximum values (`[1000, 1000]`):** `999 * 999 = 998001`. No overflow concerns in Python (integers are unbounded).
- **Mixed with a single large value (`[1, 1, 1, 100]`):** The runner-up is `1`, so the answer is `99 * 0 = 0`. A good reminder that *both* chosen numbers affect the result.

20. Common Mistakes

1. **Assuming the two values must be different.** The problem asks for two different *indices*, not two different *values*. `[2, 2]` is valid and returns `1`.
2. **Forgetting to subtract one.** Returning `nums[-1] * nums[-2]` gives `20` on `[3, 4, 5, 2]` instead of the correct `12`.
3. **Getting the single-pass swap order wrong.** Overwriting `first` *before* copying its old value into `second` silently loses the runner-up.
4. **Reusing the same index in brute force.** Starting the inner loop at `j = i` (instead of `j = i + 1`) lets a position pair with itself, which isn't allowed.
5. **Assuming negative numbers are possible.** This problem's constraints guarantee positive integers, so initializing trackers to `0` is safe — but don't blindly copy that pattern to problems that *do* allow negatives.

21. Interview Questions

Expect follow-ups like:

- *"What if the array could contain negative numbers?"* Then two large negatives might produce a large positive product — the Top-Two Grab alone breaks, and you'd also track the two *smallest* values.
- *"Can you do it in a single pass with O(1) space?"* Yes — walk once, tracking the top two.
- *"What changes if you need the top three elements?"* Extend the tracking logic, or reach for `heapq.nlargest(3, nums)`.
- *"Why is sorting O(n log n) and can you beat it?"* Yes, a linear scan finds the top two in `O(n)`.
- *"What if the array is streamed and doesn't fit in memory?"* The single-pass approach shines — it never needs the whole array at once.

22. Similar Problems

- **LeetCode 1竞 — Kids With the Greatest Number of Candies** — another "compare against the max" warm-up.
- **LeetCode 414 — Third Maximum Number** — same "track the top few" instinct, extended to three.
- **LeetCode 628 — Maximum Product of Three Numbers** — the natural next step, where negatives make it genuinely trickier.
- **LeetCode 215 — Kth Largest Element in an Array** — generalizes "find the top few" using heaps or quickselect.
- **LeetCode 1913 — Maximum Product Difference Between Two Pairs** — needs both the two largest *and* the two smallest.

Each one reuses the muscle you just built: identify which few elements actually matter, and ignore the rest.

23. Key Takeaways

- The formula rewards larger numbers, so **only the two largest values ever matter** — the Top-Two Grab.
- **Sorting** is the cleanest way to surface them; `nums[-1]` and `nums[-2]` are the two biggest.
- A **single pass** trades a little readability for `O(n)` speed.
- The `[2, 2]` case returns `1` because two identical values in different positions are perfectly valid.
- Choosing the *clearest* correct solution — not just the fastest — is itself an interview skill.

24. Watch the Video

Want to see the Top-Two Grab traced out loud, complete with the pause-and-guess `[2,2]` moment? **Watch the full walkthrough here:** {LINK}

Following along visually makes the "why it always works" click far faster than reading alone — so hit play and code it with me.

25. About the Series

This is part of **Fun with Learning Technology** — a series that turns intimidating-looking coding problems into simple, intuition-first lessons. We start every problem the way a real engineer thinks: build the mental picture first, *then* write the code. No memorizing, no hand-waving. Just clear reasoning you can reuse on the next problem, and the one after that. Tomorrow we take this same "find the top few" instinct to a slightly bigger challenge.

26. Call To Action

If this made the problem click for you:

- 👍 **Like** the video on YouTube — it genuinely helps the channel grow.
- 🔔 **Subscribe** so you never miss the next lesson in the series.
- 📩 **Subscribe on Substack** for the written deep-dives like this one.
- 💬 **Comment** with how *you* solved `[2, 2]` — did the answer surprise you?
- 🔗 **Share** this with a friend who's just starting arrays.

SOCIAL MEDIA

LinkedIn Post

Some coding problems look hard only because of how they're written.

Take LeetCode 1464 — "Maximum Product of Two Elements in an Array." The formula `(nums[i] - 1) * (nums[j] - 1)` makes beginners freeze. But the actual idea is tiny.

Here's the insight that unlocks it: subtracting one and then multiplying *rewards* bigger numbers. So the smallest values in the array can never help you. You only ever need the two largest. I call this the

Top-Two Grab.

Once you see that, the solution is two lines: 1. Sort the array. 2. Multiply `(last - 1) * (second_to_last - 1)`.

Time complexity is O(n log n), space is O(1). If you want to skip sorting, a single pass tracks the top two in O(n) — faster, but the swap logic is easy to get subtly wrong. For an easy problem, I'd sort and keep it readable.

One detail trips people up: the problem asks for two different *positions*, not two different *values*. That's why `[2, 2]` returns 1, not 4 — subtract one from each, multiply, done.

Why does a problem this small matter? Because interviewers use it to test whether you can recognize a pattern, reason about complexity, and handle edge cases calmly. And the "find the top few" instinct shows up everywhere in real systems — leaderboards, best-price selection, top-rated results.

Master the small problems cleanly, and the big ones stop feeling big.

What's your favorite deceptively-simple LeetCode problem?

Python #LeetCode #CodingInterview #Algorithms #SoftwareEngineering

Twitter/X Thread

1/ LeetCode 1464 looks scary: "return the max of `(nums[i]-1)*(nums[j]-1)`."

It's actually one of the easiest array problems out there. Let me show you why. 🧵

2/ The whole problem: pick two different spots in an array, subtract 1 from each value, multiply them, and make that result as big as possible.

That's it. No clever math required.

3/ Key insight: subtracting one and then multiplying *rewards* bigger inputs.

So the small numbers can NEVER help you. You only ever need the two largest values.

I call this the "Top-Two Grab."

4/ How do you find the two largest? The most intuitive way is sorting.

Sort smallest → largest, and the two biggest numbers are simply the last two in line. Like kids lining up for a class photo.

5/ In Python, negative indices count from the end: - `nums[-1]` = last (biggest) - `nums[-2]` = second-to-last

So after sorting, those two ARE your answer.

6/ The full solution:

```
nums.sort()
return (nums[-1] - 1) * (nums[-2] - 1)
```

Two lines. O(n log n) time, O(1) space.

7/ Quick quiz: what does `[2, 2]` return?

Subtract one from each → 1 and 1. Multiply → **1**.

Not 4! Two identical values in different positions are totally allowed.

8/ Want to skip sorting? Walk the list once, tracking the biggest and second-biggest as you go. That's O(n) — faster.

The catch: the swap logic is easy to mess up. For an easy problem, I just sort.

9/ Edge cases the trick handles for free: - All ones → 0 - Duplicates → fine - Two-element array → uses the only pair No special cases needed.

10/ The real lesson isn't this problem — it's the "find the top few" instinct.

Leaderboards, best prices, top scorers… you'll reuse it forever.

Full walkthrough (with the [2,2] pause-and-guess) 👇 {LINK}

Facebook Post

Ever looked at a coding problem and felt your brain shut down at the notation? 😅

LeetCode 1464 does exactly that — until you realize it's tiny.

The task: pick two numbers from a list, subtract one from each, multiply them, and get the biggest possible result. The secret? Bigger numbers always win, so you only ever need the two largest. Sort the list, grab the last two, do a little math — done in two lines.

And here's the fun gotcha: what does `[2, 2]` return? Not 4... it's 1! (Subtract one from each first. 😉)

I break the whole thing down step by step in the new video — beginner-friendly, no math degree required. Come learn it with me 👇 {LINK}

Reddit Summary

Breaking down LeetCode 1464 (Maximum Product of Two Elements) — the intuition, not just the code

This one looks harder than it is because of the `(nums[i]-1)*(nums[j]-1)` formula. Here's the reasoning that makes it click:

Since you subtract one and then multiply, bigger inputs always give a bigger result. That means the smaller numbers in the array are irrelevant — you only ever need the two largest values.

Two clean approaches: - **Sort:** `nums.sort()` then `(nums[-1]-1) * (nums[-2]-1)`. O(n log n), very readable. - **Single pass:** track the top two as you scan. O(n), but the swap logic is easy to get subtly wrong.

The detail worth flagging: the problem wants two different *indices*, not two different *values*. So `[2, 2]` returns `1` (1×1), not `4`. That catches a lot of people.

Edge cases (all ones → 0, duplicates, two-element arrays) all work without special handling.

The takeaway that generalizes: recognizing "I only need the top few elements" is a pattern you'll reuse in problems like 628 (Max Product of Three) and 215 (Kth Largest). Curious how others approached the single-pass version — did anyone bother with a heap for just k=2?

GITHUB README

```
# Maximum Product of Two Elements in an Array — LeetCode 1464

> Beginner-friendly. Part of the **Fun with Learning Technology** series.

## Problem

Given an array of integers `nums`, choose two **different indices** `i` and `j`
and maximize the value of `(nums[i] - 1) * (nums[j] - 1)`.

- `2 <= nums.length <= 500`
- `1 <= nums[i] <= 1000`
```

````
**Note:** the two indices must be different, but the *values* at them may be equal.

### Examples

| Input            | Output | Why                          |
|------------------|--------|------------------------------|
| `[3, 4, 5, 2]`   | `12`   | `(5-1) * (4-1) = 4 * 3`      |
| `[1, 5, 4, 5]`   | `16`   | `(5-1) * (5-1) = 4 * 4`      |
| `[2, 2]`         | `1`    | `(2-1) * (2-1) = 1 * 1`      |

## Intuition

Subtracting one and then multiplying **rewards larger inputs**. So the smallest
numbers can never contribute to the answer — you only ever need the **two
largest** values. Call it the *Top-Two Grab*.

## Approach

1. **Sort** the array ascending — the two largest values move to the end.
2. Take `nums[-1]` (largest) and `nums[-2]` (second largest).
3. Subtract one from each and multiply.

An alternative single pass tracks the top two in one scan for O(n) time.

## Python Solution

```python
from typing import List


class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        nums.sort()
        return (nums[-1] - 1) * (nums[-2] - 1)
````

▶ Single-pass O(n) version

Complexity

Approach	Time	Space
Brute force	`O(n^2)`	`O(1)`
Sort	`O(n log n)`	`O(1)`
Single pass	`O(n)`	`O(1)`

Video

▶️ Watch the full walkthrough: {LINK}
````

Article

📖 Read the in-depth explanation with dry runs, edge cases, and common mistakes: {LINK}

⭐ Star this repo if it helped, and check out the rest of the series. ```

Quick Review

Pattern: find max product of two elements. What's the trigger?

When asked for the best pair (max/min product or sum) and only the two extremes matter, think: just grab the two largest values — no need to test all pairs.

Time complexity to find the two largest elements?

O(n) — one pass tracking the top two values. Sorting first would be O(n log n), which is unnecessary here.

Space complexity for the single-pass approach?

O(1) — just two variables holding the largest and second-largest seen so far.

Common bug when tracking the top two values?

Updating second-largest wrong: when a new max appears, the old max must become second. Also this problem subtracts 1 from each, so answer is (a-1)*(b-1).

Brute force vs single-pass for the max pair?

Brute force checks every pair: $O(n^2)$. Single pass keeps only the two biggest: O(n). Same answer, far fewer comparisons.

Follow-up: what if numbers could be negative?

Two large negatives can give a bigger product than two positives. Track the two smallest and two largest, then compare both products.
